Grayson Gilbert
ENPM690
Final Project Report

Robust Control of a Furuta Pendulum:

Leveraging PPO and MuJoCo for Sim2Real Transfer

Abstract:

This project demonstrates the successful zero-shot sim2real transfer of a deep reinforcement learning policy for the robust control of a furuta pendulum. The furuta pendulum, a highly non-linear, 2-Degree of Freedom (DoF) underactuated robotic system, presents a complex continuous control challenge due to real-world friction, sensor noise, and communication latency. To effectively bridge the simulation-to-reality gap, a high-fidelity MuJoCo physics simulation was developed and meticulously calibrated against physical hardware responses using isolated drop and rotor tests. A Proximal Policy Optimization (PPO) agent was trained within a custom Gymnasium environment that utilized a multi-phase curriculum learning reward structure to sequentially teach swing-up and balancing behaviors. To ensure policy robustness upon deployment, the training environment heavily incorporated domain randomization, manual velocity noise injection, and action latency buffers. The finalized model weights were deployed directly to an ESP32 microcontroller leveraging Torque Field Oriented Control (FOC), requiring absolutely no further tuning or training on the physical hardware. The physical system successfully executed the swing-up maneuver and maintained an active balance for over a continuous hour during endurance testing. Ultimately, this work highlights the efficacy of combining accurately grounded physics simulations with robust reinforcement learning strategies to overcome the sim2real gap in complex, underactuated physical systems.

Project Goal/Introduction:

The goal for this project is to bridge the gap between high-fidelity physics based simulation and physical hardware by developing a robust Proximal Policy Optimization (PPO) based controller for a furuta pendulum. The project aims to train an agent that can repeatedly swing up and balance a furuta pendulum, while handling the noise, friction, and non-linearities of the real world.

The furuta pendulum, also known as the rotary inverted pendulum, is a classic 2 Degrees of Freedom (DoF) underactuated control problem. It is composed of a single, powered, rotating base, attached to a free-spinning unpowered pendulum mass.

An example of a furuta pendulum controlled via classical control methods can be found by clicking the following link: Furuta Pendulum Example Video

Literature Survey:

Proximal Policy Optimization (PPO) in Continuous Control: Deep reinforcement learning (RL) has quickly become the go-to approach for continuous control tasks in robotics. For this project, Proximal Policy Optimization (PPO) was selected because it strikes an excellent balance between sample efficiency and training stability. Research shows that PPO improves upon earlier Trust Region Policy Optimization (TRPO) methods by utilizing a clipped surrogate objective. This clipping mechanism prevents the model from making excessively large policy updates during training. As a result, it maintains performance guarantees while significantly reducing computational overhead, making it highly effective for the high-frequency 100Hz control loops required when running on the physical microcontroller.

Reinforcement Learning for the Furuta Pendulum: The Furuta pendulum is characterized by highly non-linear dynamics driven by gravity, Coriolis, and centripetal forces. Traditionally, controlling this system required complex analytical derivations and heavy simplifications of physical laws to keep the pendulum in an inverse equilibrium state. However, recent advancements demonstrate that RL algorithms can successfully bypass these explicit rigid-body modeling constraints. By allowing the agent to learn adaptive control policies directly within high-fidelity physics simulators, like MuJoCo, stable policies can be developed rapidly without the temporal or spatial constraints of training on physical hardware.

Bridging the Sim2Real Gap through Robust Control: While simulation environments allow for rapid and safe policy exploration, transferring these trained policies to the real world remains the largest hurdle. Optimal policies generated in pure simulation frequently overfit to the approximated dynamics and fail when exposed to real-world uncertainties like sensor noise, communication latency, and unmodeled friction. Robust reinforcement learning addresses this fragility by explicitly accounting for these uncertainties during the training phase. By introducing external disturbances and randomizing physical parameters (like mass, motor torque, and damping) within the training environment, the agent is forced to learn resilient recovery behaviors. This approach yields policies that are significantly more robust against physical mass changes and non-ideal actuators.

Summary and Research Implications: Overall, the literature confirms that while model-free RL algorithms like PPO are highly capable of generating successful policies for underactuated systems, raw sim-trained models are inherently brittle upon physical deployment. Overcoming this sim2real gap requires the explicit integration of robust RL techniques like domain randomization, noise injection, and structured curriculum learning. These foundational findings directly informed the methodology of this project, justifying the design of the custom Gymnasium environment to ensure the PPO agent learns predictive, robust control capable of seamless transfer to the physical system.

Hardware:

Component Breakdown:

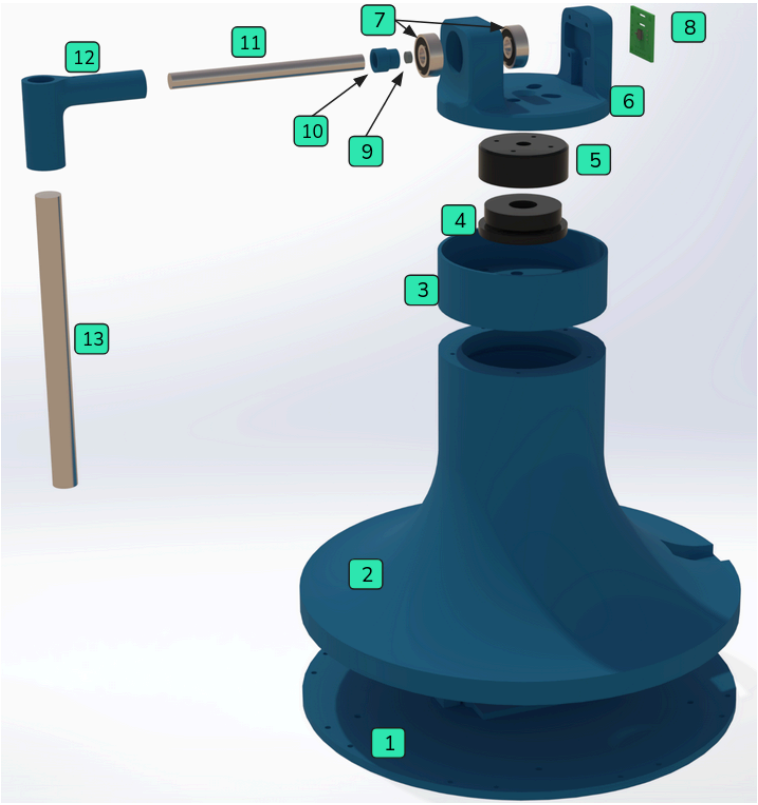
- 3D Printed Pendulum Parts (base, pendulum support, and other various components)
- 1x iPower GM4108H-120T Brushless Gimbal Motor
- 1x 8mm slip ring
- 1x CUI Devices AMT103V ABI Encoder
- 1x I2C Bi-directional 3.3V-5V Level Shifter
- 1x ESP32 Lolin Lite Development Board
- 1x DENG FOC V3 Brushless Driver Board
- 1x Push-button Switch Development Board
- 1x AS5047P SPI Magnetic Rotary Encoder
- 2x 304 Stainless Steel Rods for Pendulum Mass
- 1x Benchtop Power Supply

CAD Model:

The 3D model was originally developed using Solidworks. Each individual model was designed from scratch, except for the AS5047P encoder, which was provided by the board manufacturer. Once the individual components were complete, the entire system was assembled, and constrained accordingly. It is important to note that at this step all component frames must be aligned appropriately before exporting the model for simulation. Failure to do so will result in misaligned parts and unexpected responses from the simulated physics engine. An exploded view of the exported assembly can be found below. In an effort to simplify the model during simulation, non-essential electronic components were removed from the assembly.

Exploded View Diagram:

1. PLA Pendulum Base - bottom
2. PLA Pendulum Base - middle
3. PLA Pendulum Base - top
4. BLDC Stator
5. BLDC Rotor
6. Rotating Pendulum Support
7. Ball Bearing
8. AS5047P SPI Encoder
9. Diametric Magnet
10. Magnet Holder
11. 8mm SS Rod
12. Pendulum Rod Holder
13. 12mm SS Rod



Exploded view of pendulum assembly.

Model Simulation:

Platform:

The pendulum model was then exported to the MuJoCo physics engine for simulation. MuJoCo stands for Multi-Joint Dynamics with Contact, and it is an open-source physics engine developed by Google Deepmind. MuJoCo was selected due to its performance, accuracy, and lightweight nature allowing for high-fidelity simulation on my host machine. Other notable features that aided in the selection of this platform include domain randomization during training, high-frequency training while maintaining a constant control loop rate, and mechanical fidelity between joints and components. These features allow for an accurate simulation of the furuta pendulum without the need for expensive industrial grade GPUs.

Model Setup:

Once the Solidworks assembly was complete, it needed to be exported into a format MuJoCo can work with. Solidworks is unable to export assemblies in the MuJoCo native MJCF format, however it can export files as a URDF (Unified Robot Description Format). A python script was then developed to convert the URDF to the MJCF for processing in MuJoCo. The base model exported from Solidworks contains the 3D meshes, joints, and links necessary for simple simulation, but it lacks many of the details needed for the realistic physics based simulation.

After the base model was successfully imported into MuJoCo, system specific information was added to the model to ground the simulation in reality.

Notable motor specifications were pulled directly from the manufacturer's website linked here.

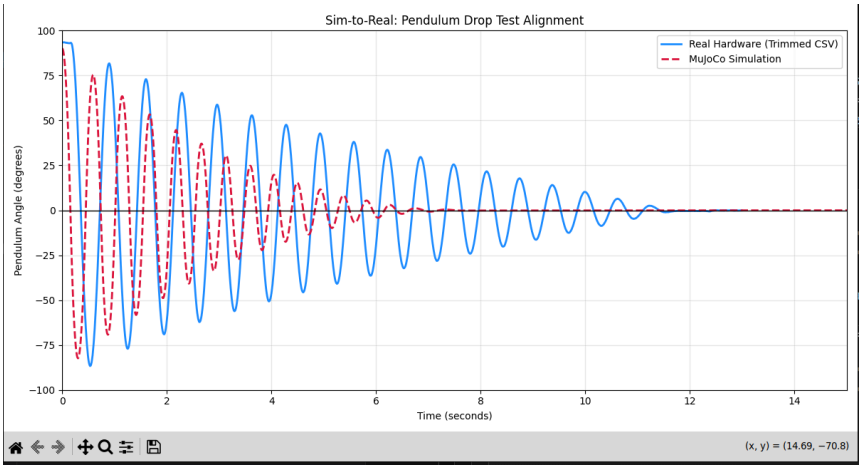
No-load Voltage: 20V
No-load RPM: 513-567RPM [540RPM avg]
Load Voltage: 20V
Load current: 1.5A

From these specifications the following information about the motor can be calculated:

Kv (Motor Velocity Constant) = No-load RPM / No-load Voltage = 540/20 = 27
Torque constant = 8.27 / Kv = 8.27 / 27 = 0.306 Nm/A

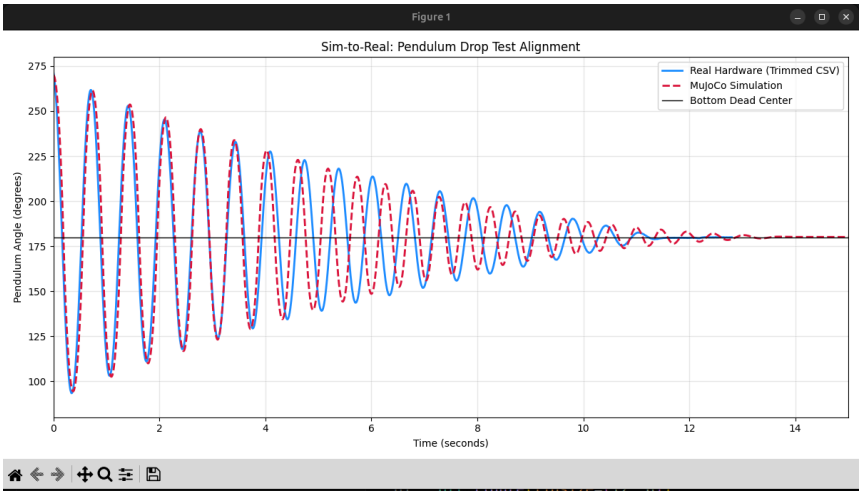
The load current defines the control range of the actuator to [-1.5A, 1.5A], and the torque constant (0.306) is directly applied to the actuator gear value in the MuJoCo model. Additionally, position and velocity sensors at the rotor and pendulum joint were defined to replicate the physical system hardware.

After the physical motor parameters were established, the remaining simulation parameters were tuned to match the physical system response. First, an isolated pendulum drop test was performed. The rotor of the BLDC was fixed in place to completely isolate the pendulum motion. The pendulum was raised to be 90 degrees from the bottom resting position. At this point a data collection logger was started to record pendulum position over time. The pendulum was released from the horizontal position and allowed to oscillate back and forth until coming to a complete stop. The experiment was repeated in the MuJoCo simulation and the results were compared.



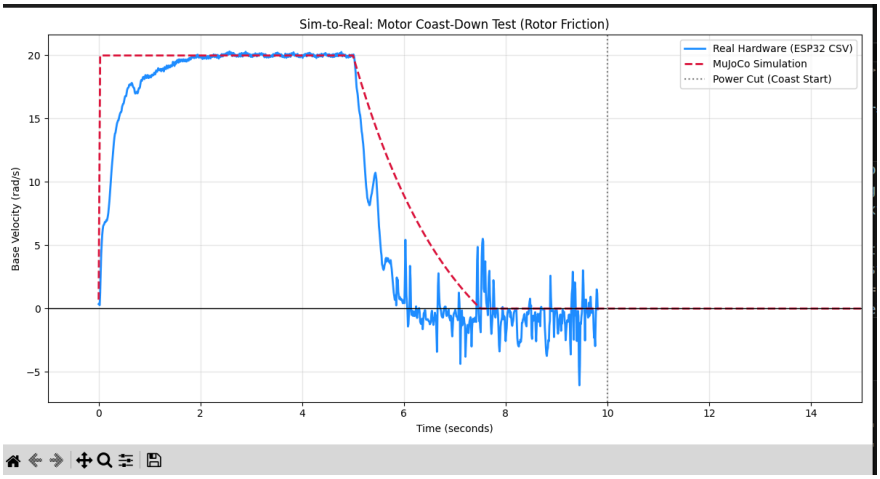
Initial pendulum drop test response.

The initial pendulum drop test showed that phase of the simulated pendulum oscillations did not match the physical system. This was likely due to the simulated center of gravity differing from the real system. To resolve this, I modified the pendulum center of mass location in the MuJoCo model until the phase was better aligned with the real system. The damping and friction loss parameters were also tuned until the simulated pendulum nearly matched the physical response shown below.



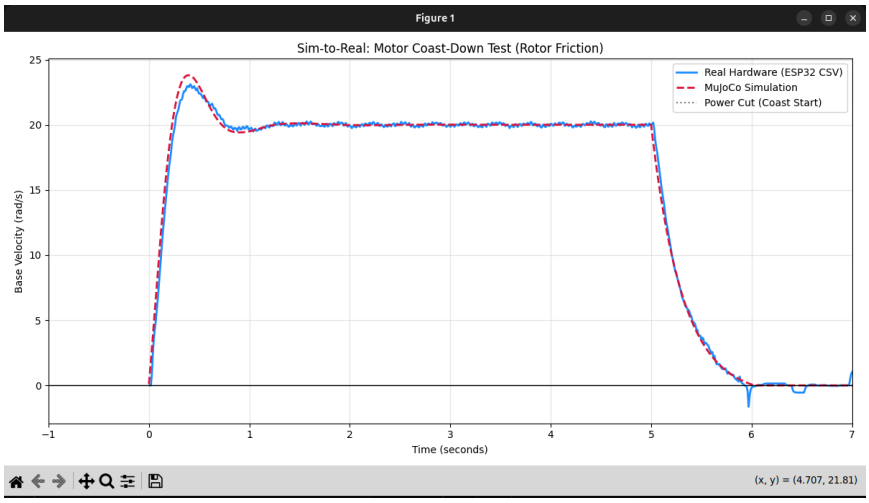
Final pendulum drop test response.

After the pendulum simulated response matched the physical system, the rotor isolation test was performed. For this test the pendulum was fixed in the bottom position, and the rotor was commanded to spin at constant rate. Like the first test, a logging script was deployed to record system data. The rotor isolation test started the motor from 0 rad/s and accelerated it until it reached a steady 20 rad/s. This velocity was held constant for 5 seconds before the motor commands were stopped and the rotor was allowed to coast until it came to rest. The same experiment was repeated in the simulation and the initial test results are shown below. Please note that for this initial experiment, the pendulum was not constrained appropriately resulting in noisier readings, and a different initial response than in the final tests.



Initial rotor isolation test response.

After conducting the initial rotor isolation tests, the friction loss and damping values were once again tuned until the simulation better aligned with the physical system response. The final rotor isolation response is shown below.



Final rotor isolation test response.

At this point the MuJoCo model was grounded in physics, and the simulated pendulum system response was nearly identical to the physical pendulum system. The model was now ready for the next step, which was creating the environment for training.

Environment Setup:

With the MuJoCo model ready for training, the next step was to create the simulation environment for reinforcement learning. To achieve this, I wrapped the MuJoCo model in a custom Gymnasium environment. Gymnasium is a reinforcement learning API that bridges the gap between the MuJoCo physics engine and the RL algorithm. The environment consists of the following functions: initialize, step, observe, terminate, reset, and reward.

Initialize:

The environment initializes by first loading in the MuJoCo model data. After this, it will define a frame skip variable and set it equal to 10. The frame skip is used to mimic the real world 100Hz control loop that runs on the microcontroller. A frame skip of 10 results in a control loop rate of 100Hz because the MuJoCo model timestep is 0.001 seconds. After the frame skip is defined, physical hardware limits like motor current and encoder resolution are set. Then the 1 dimensional action space is initialized and constrained to the range of [-1, 1]. For this environment, the action space is the commanded motor current. Next, the 6 dimensional observation space is defined and constrained to the range of [-1, 1]. Various model parameters like motor torque constant, model damping, model friction, and model mass are then initialized for future domain randomization in the reset function. Finally, the action delay buffer is initialized to mimic real life communication latency.

Step:

This function is also known as the execution loop for the RL environment. The simulation step takes the model action as a parameter, in this case the commanded motor current. Because the action is constrained to a range of [-1, 1], it is multiplied by the maximum motor current to scale the action into physical amps. Next the action is added to the delay buffer so it does not get applied immediately. Instead the model must wait a couple steps before the action is applied. This buffer mimics the real life communication latency in the system, and forces the model to learn predictive control instead of relying on instantaneous response times. Then, the commanded current for this given step is applied for the next 10 frames. This is how the frame skip is implemented, and sets the 100Hz control rate. After the frame skips are complete, the raw position and velocity data for the motor and pendulum are read from the model. The pendulum position is then wrapped between the values of [-pi, pi], and the raw system state is generated. The observation function is called using this raw state as an input, followed by the reward function. Next, the previous target current is updated to be the current target current, and the termination function is checked to see if termination criteria are met. If the simulation is to be terminated, the type of termination is logged for telemetry. Finally, the observation, reward, termination, and telemetry values are returned.

Observation:

The observation function takes the raw system state as an input parameter. The raw state includes rotor position, rotor velocity, wrapped pendulum position, and pendulum velocity. Then the raw state positions are quantized into acceptable values within the encoder resolution. Next, noise is manually injected into the velocity sensor readings. This noise is added because of the way the physical system calculates the pendulum and rotor velocity. In the real system, the velocity is calculated by taking the derivative of the encoder position. These sensor readings are inherently noisy, which can have a large impact on overall velocity when the motor or pendulum is moving at slow speeds. The simulation sensors do not automatically have noise in their readings, so it must be injected manually. Next the maximum rotor position, maximum rotor velocity, and maximum pendulum velocity are defined. These values are used in the following observation array to keep the velocity and position values between the range [-1, 1]. The observation space array includes the following values:

1. Discrete rotor position / maximum rotor positon
2. Noisy rotor velocity / maximum rotor velocity
3. Sine component of discrete pendulum position
4. Cosine component of discrete pendulum position
5. Noisy pendulum velocity / maximum pendulum velocity
6. Previous commanded current / maximum commanded current

The pendulum position is broken down in its sine and cosine components as a way to prevent sudden value jumps when the pendulum wraps around the maximum position. These sudden jumps in value would otherwise confuse the RL model during training, resulting in poor performance.

Reset:

The simulation reset occurs when training starts, or when termination criteria are met. The reset function completely resets the training environment, preparing it for the next episode of training. The pendulum environment contains two potential training modes that affect the system reset. The training modes are Balance and Swing Up. The Balance mode resets the pendulum environment and initializes the pendulum near the Top Dead Center (TDC) position. This is the upright, unstable position. The Swing Up mode resets the pendulum environment and initializes the pendulum in the Bottom Dead Center (BDC) position. Both training modes also incorporate domain randomization. This means that the simulation randomizes the values for motor torque, model friction, model damping, and model mass within a specified range. This domain randomization helps to improve overall model performance and robustness.

The various training modes are a result of curriculum learning. For more information about the curriculum learning process please see the Reward function section below.

Termination:

The termination criteria for the simulation episode is based on the mode of the training being conducted. If any of these criteria are met during a training episode, the simulation will immediately stop the episode, log the termination type, and reset the environment.

The termination criteria for the Balance training mode is the following:

- The pendulum drops below 15 degrees away from the TDC position
- The motor spins more than 2 full rotations in any one direction

The termination criteria for the Swing Up training mode is the following:

- The motor spins more than 2 full rotations in any one direction

Reward:

The reward function is the most important part of the training environment. Without an appropriately designed reward, the model will not be able to learn the desired task. Swinging the pendulum from the BDC position, and then catching and balancing it in the TDC position is not trivial. It would be very difficult to design a reward function that can achieve this system response without breaking it up into smaller parts. This is where curriculum learning comes into play. Instead of starting model training by attempting difficult and complex tasks, the curriculum learning approach breaks the complex problem down into smaller more manageable tasks. The agent is taught to successfully complete the smaller, easier tasks before learning more challenging operations. This approach is very similar to how students first learn basic arithmetic before tackling harder concepts like calculus.

With this in mind, the pendulum problem was broken down into two main sub-tasks: 1. Swing up the pendulum 2. Catch and balance the pendulum. The second task was the easier of the two so that is where the training started. The model learned to stabilize the pendulum in the upright position using the Balance training mode. Each episode reset the pendulum near the TDC

position allowing for the model to have millions of opportunities to learn to balance the unstable mass. Once the reward was designed in a way that enabled the model to successfully balance the pendulum, the swing up task was taught. The swing up task was trained using the Swing Up mode. It built upon the current reward structure to successfully generate momentum to swing the pendulum from the BDC position up to TDC and balance it there for extended periods of time. The final reward function was broken down into two distinct phases based on pendulum position with a third transition phase blending everything together.

The overall reward function is made up of a variety of rewards, bonuses, and penalties. A high level overview of all the rewards, bonuses, and penalties are as follows:

- Upright reward = cos(pendulum position) - 1
- Effort penalty = 0.001 * (current ** 2)
- Centering penalty = 0.03 * (rotor position ** 2)
- Action rate penalty = 0.5 * ((current -previous current) ** 2)
- Rotor velocity penalty = 0.05 * (rotor velocity ** 2)
- Catch velocity penalty = 0.02 * (pendulum velocity ** 2) + 0.05 * (rotor velocity ** 2)
- Momentum bonus = 0.1 * abs(pend. Velocity), values clipped between [0.0, 0.8]
- Swing weight = 1.0 - catch weight
- Blended Momentum Bonus = momentum bonus * swing weight
- Blended Velocity Penalty = (swing base velocity penalty * swing weight) - (catch velocity penalty * catch weight)
- Catch Bonus = 3.0 if catch weight == 1.0, otherwise 0
- Duration Bonus = 0.01 * consecutive upright steps

The total reward is defined at the sum of all rewards and bonuses minus the sum of all penalties.

$$\text{Total Reward} = \left[\text{(upright reward + catch bonus + blended momentum bonus + duration bonus)}\right] - \left[\text{(blended vel. penalty + centering penalty + effort penalty + action rate penalty)}\right]$$

Swing Up Phase Reward Structure:

In the Swing Up phase, reward structure promotes generating pendulum momentum, in order to eventually swing the pendulum up to the TDC position. The pendulum is in the phase any time the pendulum position is greater than 0.6 rads away from TDC. The Swing Up phase reward structure is the following:

- Catch weight = 0
- Swing weight = 1
- Blended momentum bonus -> has full momentum bonus effect
- Blended velocity penalty -> only swing base velocity penalty, no catch penalties
- Catch bonus = 0

- Consecutive upright steps = 0
- Duration bonus = 0

Transition Phase Reward Structure:

The distance between 0.6 rads away from TDC and 0.2 rad away from TDC is considered the transition phase of the reward structure. This space is where both the Swing Up and Balance phases blend together to smoothly transition from one reward structure to the next. The transition phase also helps to slow down the pendulum momentum generated in the Swing Up phase so the pendulum does not carry over the TDC position and swing back towards the BDC position. The transition phase reward structure is as follows:

- Catch weight = (1.2 - distance from TDC) / 1
- Swing weight = less than 1
- Blended momentum bonus -> has reduced momentum bonus effect
- Blended velocity penalty -> contains some swing base velocity penalty and some catch penalties
- Catch bonus = 0
- Consecutive upright steps = 0
- Duration bonus = 0

Catch and Balance Phase Reward Structure:

When the pendulum position is less than 0.2 rads from TDC the Catch and Balance reward phase kicks in. Here the reward heavily promotes keeping the pendulum in this upright position as close to TDC as possible. The Catch and Balance reward structure is as follows:

- Catch weight = 1
- Swing weight = 0
- Blended momentum bonus = 0
- Blended velocity penalty -> swing base velocity penalty + catch velocity penalty
- Catch bonus = 3
- Consecutive upright steps -> +1 for each consecutive step in this phase
- Duration bonus -> greater than 0, and increasing

These different phases of a single reward function allow for one model to be able to learn multiple unique tasks in order to achieve the goal of swinging up, catching, and balancing the pendulum in an unstable position.

Logging:

Throughout the entire simulation process, the observation space of the simulated pendulum is

logged. This allows for detailed evaluation and comparison of the simulated and physical pendulum system.

Reinforcement Learning:

Algorithm/Approach:

The control strategy relies on Proximal Policy Optimization (PPO), a stochastic policy gradient method chosen for its balance of sample efficiency and stability in continuous action spaces. The overall training pipeline is divided into three distinct phases to maximize the likelihood of success when transferring to real hardware: 1. Initial policy optimization using PPO 2. Improving robustness through randomization 3. Hardware deployment and validation.

The Initial policy optimization using PPO is where the model will learn to swing up and balance the pendulum, with a reward function that promotes smooth motor torque and extended pendulum balance times. The second phase is where the model will incorporate noise into the sensors, latency in motor response time, and changes in physical constants such as mass , motor torque, damping, and friction. This will force the model to create a control response that can withstand the randomness that is the real world. The final phase will be deploying the model on the hardware and validating its effectiveness through balancing the pendulum.

Training Architecture:

The training program leverages Stable Baseline 3, an open-source python library that provides reliable implementations of the Deep Reinforcement Learning algorithms using the PyTorch framework. More specifically, the training program uses the PPO model built into SB3. The training can be initialized using one of two modes, the Balance mode or the Swing Up mode. Each mode corresponds to the initial environment conditions described in the custom gymnasium environment above. When training is initialized, it will start 10 parallel processing instances of the pendulum model. Parallelizing the model training enables significantly more training capacity without increasing overall training time. The training program also leverages SB3's extensive logging capabilities with Tensorboard. In addition to the built in logging features, a custom termination callback was implemented to track the number of each termination type as the training evolves. These logging features allow for model evaluation without seeing the simulated system response of the pendulum model. For periodic model evaluation, the training program saves checkpoint policies every 10,000,000 steps to get a better understanding of how the agent is learning without stopping training completely. A single training episode can have a maximum of 1000 timesteps before restarting. This helps to prevent the model from wasting time learning ineffective strategies. The training will proceed for a total of 30,000,000 steps split amongst all parallel instances before saving the final model policy.

Model Hyperparameters:

- Learning rate: 0.0003
- Number steps: 2048

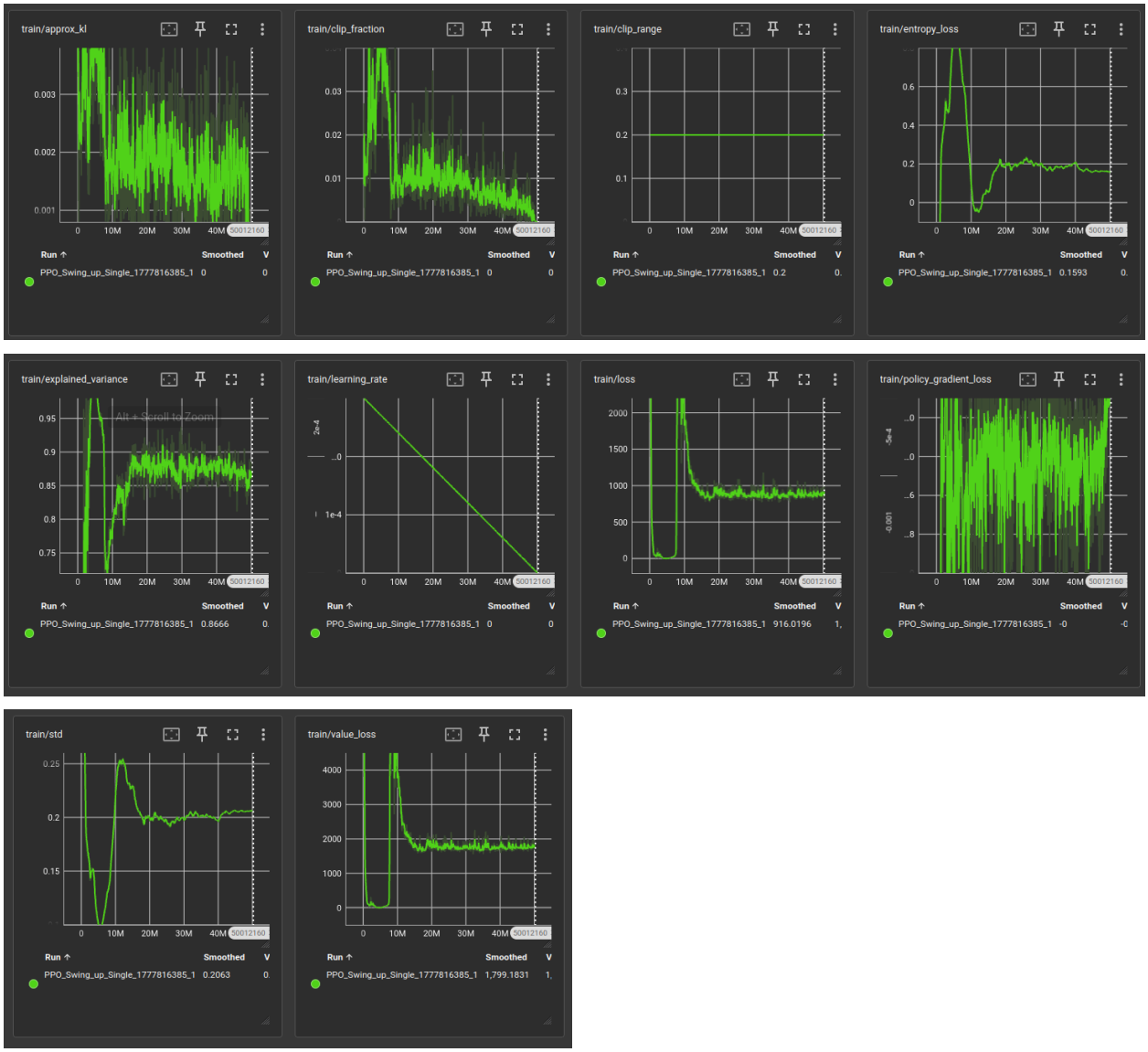
- Batch size: 2048
- Number epochs: 10
- Entropy coefficient: 0.03
- Clip range: 0.2

Simulation Results:

Following the completion of model training on the pendulum simulation, the model was able to successfully swing up and balance the pendulum in the TDC position. A recording of the simulation in action can be found below:

Simulation Demonstration

Below are screenshots of the Tensorboard logs saved from the simulation training process. The first 4 charts are the logs that were most frequently used when evaluating the success of a model during training. Starting from the top left, the base drift termination chart plots how frequently the simulation ended prematurely due to the base drift termination criteria. The next chart shows the pendulum drop criteria. The Swing Up training mode was performed for this simulation, so pendulum drop criteria is ignored. The next chart shows the average reward of a simulation. The reward initially started out very negative as the pendulum was heavily penalized for improper control of the pendulum. As the model learned to generate momentum to swing the pendulum, the reward function slowly increased until it reached 0. When the model finally caught and balanced the pendulum for the first time, the reward sharply increased to the +6000 range where it remained for the rest of the simulation. The top right chart shows the average length of a simulation episode. Each episode can have a maximum of 1000 timesteps before termination. A successful simulation should be able to reach the maximum simulation timestep without terminating. As you can see, this simulation approached the maximum 1000 timestep limit once the model learned to balance the pendulum successfully.



Tensorboard logging charts for model evaluation.

Embedded Software:

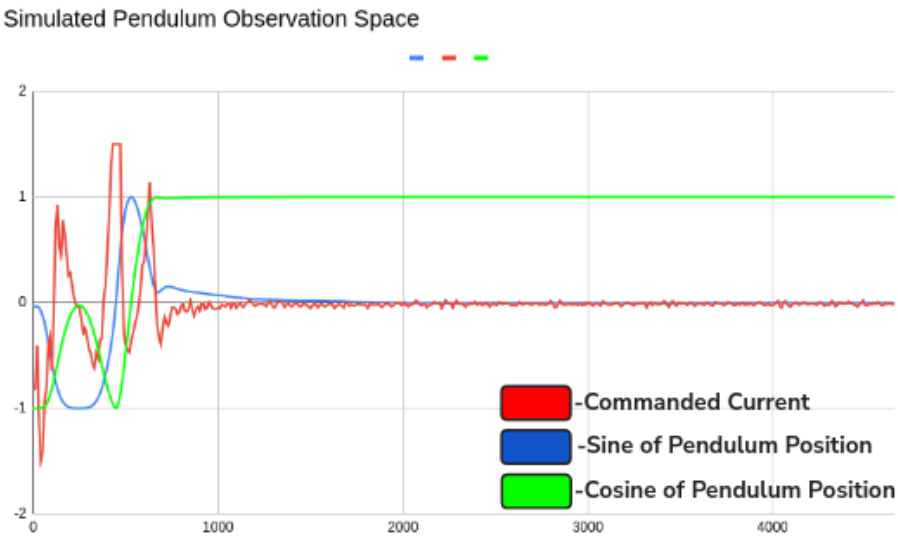
Software Architecture:

Once the simulation was successful, the process of transitioning the model onto an embedded device could take place. A completed simulation outputs the trained model in a zip file containing JSON data files, PyTorch variable files, policy files, and policy optimizer files. This zip file cannot directly run on an embedded device. Instead the trained policy weights needed to be exported into a C++ header file, so they can be compiled into the machine code running on the ESP32.

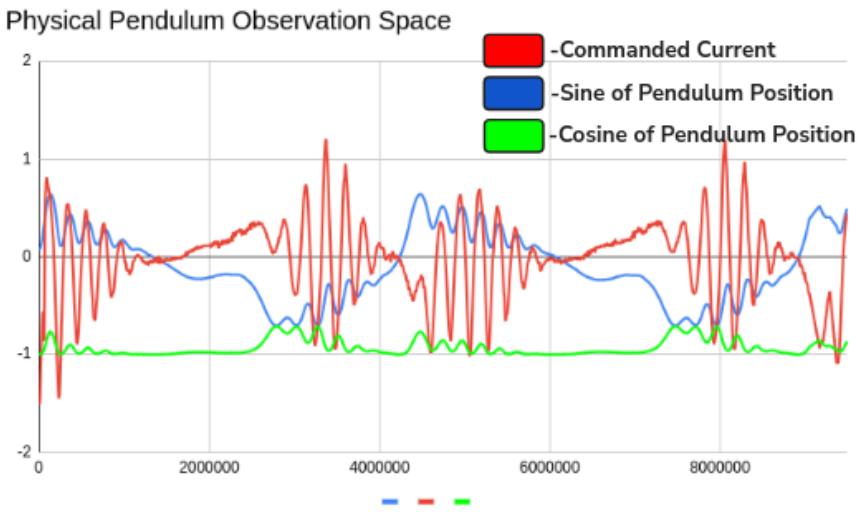
The embedded software architecture utilized both processing cores in the ESP32 microcontroller. Core 0 handled all RL processing related tasks. This core read in the pendulum observation space and calculated the required target current using the exported model weights. This control loop was fixed at the planned 100Hz rate to ensure that the commanded current was received when the model expected it. Core 1 handled all the motor control commands. The brushless motor that drives the pendulum system is controlled using Torque Field Oriented Control (FOC) via current sensing. FOC is a high performance control technique for driving Brushless DC motors (BLDC). It allows the motor to be controlled with extreme precision. The FOC control code leverages the open-source library Simple-FOC for the algorithm implementation as well as ESP32 specific firmware drivers. Getting into the weeds of the FOC control algorithm is outside the scope of this project.

Logging and Debug:

During the first system startup, the pendulum, surprisingly, did not work. In an attempt to understand what was wrong in the embedded software, observation space logging was implemented. Because the ESP32 has limited memory, the log information was recorded and sent over the serial line. As the system continuously failed to balance the pendulum, a python script was listening on the ESP32 COM port and recording the messages into a .csv file. The physical system observation space could then be compared to the simulated observation space to determine where it all falls apart.



Simulated pendulum observation space.



Physical pendulum observation space.

When comparing the physical system observation space against the simulated system observation space, it became clear that the physical system commanded current (red line) would drive the sine of the pendulum position (blue line) in the wrong direction. This was caused by an inversion in the pendulum encoder installed on the physical system. This inversion was quickly remedied with a sign change in the embedded code, but it is a great reminder why detailed logging is important in physical systems. The rich debugging tools available in simulation are not always available in real hardware.

Sim2Real Transfer Results:

Once all the bugs in the embedded software were resolved, the model was able to successfully transfer from simulation to physical hardware without any further tuning or training. The goal of the project was achieved and the simulation to reality gap was successfully crossed. Video demonstrations of the physical pendulum balancing can be found below.

Physical Pendulum Demonstration

Additionally, the upper limits of the pendulum control model were challenged with an endurance test. To achieve this, a timelapse video was set up to record the pendulum balancing with a stopwatch playing in the background. The goal was to see how long the pendulum could balance before it fell over. After 1 hour of continuous balancing, the pendulum had still not fallen. At this point the challenge was considered a success, and the system was powered off. The results show that the pendulum can certainly balance for at least 1 hour without dropping the pendulum, however it is likely that the pendulum can balance for well over one hour. A timelapse video showcasing the challenge is linked below.

1 Hour Endurance Test

Contribution:

Key repositories used in this project are:

- MuJoCo
- Gymnasium
- Stable Baseline 3
- Numpy
- Imageio
- Pandas
- Matplotlib
- Arduino Libraries:
    - Simple FOC
    - Simple FOC Drivers
    - ESP32HWEncoder
    - SPI
    - AS5047P

Without the open source projects listed above, this project would not have been possible. This project is licensed under the MIT license, allowing anyone to access and build on the work I have started.

My own personal contributions for this project involve the following:

- Designed 3D model of furuta pendulum system in Soldiworks
- 3D printed pendulum model, and prototyped physical system
- Developed and soldered pendulum electrical control circuit involving ESP32, buttons, encoders, and brushless motor
- Designed MuJoCo system model for simulating virtual pendulum system
- Calibrated MuJoCo through numerous physical system experiments
- Developed custom Gymnasium environment to perform reinforcement learning training with MuJoCo model
    - The training environment included custom termination, observation, reset, step, and reward functions
- Leveraged Stable Baseline 3 to develop a training formula for agent learning
- Developed various helper scripts to convert files to useable formats, compare simulated and physical test data, evaluate trained models, and more
- Developed embedded software to run trained model on the microcontroller and control brushless motor using Field Oriented Control techniques
- Flashed microcontroller and debugged physical system ultimately achieving the project goal of successful simulation to reality transfer enabling the control of a furuta pendulum via RL model

Conclusion:

In conclusion, this project successfully bridged the gap between high-fidelity physics simulation and physical hardware by developing a robust Proximal Policy Optimization (PPO) controller for a Furuta pendulum. By meticulously calibrating a MuJoCo simulation to match the real-world dynamics of the system and incorporating domain randomization, the trained agent achieved seamless sim2real transfer without requiring further hardware tuning. A significant contribution of this work lies in the development of a custom Gymnasium environment that leveraged curriculum learning to effectively break down the complex control problem into manageable swing-up and balance sub-tasks. Furthermore, the deployment of this reinforcement learning model on an ESP32 microcontroller utilizing Torque Field Oriented Control (FOC) resulted in highly stable physical performance, demonstrated by the system's ability to balance continuously for over an hour. While the study successfully mitigated physical constraints like sensor noise and communication latency through proactive simulation modeling, future research could focus on optimizing the embedded software architecture to better accommodate microcontrollers with limited memory or expanding the curriculum learning framework to other complex underactuated robotic systems. Ultimately, by releasing the project under an open-source MIT license, this work provides a verified, accessible foundation for future advancements in reinforcement learning and applied robotics.

References:

Papers:

Hong, M. R., Kang, S., Lee, J., Seo, S., Han, S., Koh, J.-S., & Kang, D. (2023). Optimizing reinforcement learning control model in Furuta pendulum and transferring it to real-world. *IEEE Access*, *11*, 95195–95200.

Lutter, M., Mannor, S., Peters, J., Fox, D., & Garg, A. (2021). Robust value iteration for continuous control tasks. *Robotics: Science and Systems XVII*.

Meng, W., Zheng, Q., Pan, G., & Yin, Y. (2023). Off-policy proximal policy optimization. *Proceedings of the AAAI Conference on Artificial Intelligence*, *37*(8), 9162–9170.

Moos, J., Hansel, K., Abdulsamad, H., Stark, S., Clever, D., & Peters, J. (2022). Robust reinforcement learning: A review of foundations and recent advances. *Machine Learning and Knowledge Extraction*, *4*(1), 276–315.

Wang, Y., He, H., Tan, X., & Gan, Y. (2019). Trust region-guided proximal policy optimization. *arXiv*.

Websites (Project Dependencies):

GM4108H-120T Gimbal Motor:
https://shop.iflight.com/ipower-motor-gm4108h-120t-brushless-gimbal-motor-pro217

Simple FOC Project: https://simplefoc.com/
MuJoCo Project: https://github.com/google-deepmind/mujoco
Gymnasium Project: https://gymnasium.farama.org/index.html
Stable Baseline 3 Project: https://stable-baselines3.readthedocs.io/en/master/index.html
Imageio Project: https://imageio.readthedocs.io/en/stable/#
Pandas Project: https://pandas.pydata.org/
MatPlotLib Project: https://matplotlib.org/
Numpy Project: https://numpy.org/
Arduino Project: https://www.arduino.cc/

Videos:

Furuta Pendulum Example Video: https://youtu.be/XKzzWe15DEw?si=Pk6LUR-PZWryq1gJ